前書き

データベースという言葉を知ったのは，大学 3 年生の授業であった．当時，工学部情報科学科の学生であった私は，必修科目である「データベース」の講義を受けていたが，他の授業と同様，ただ漫然と講義を受けていた．単位を修得をしたものの，データベースの意義を理解していなかった．

その後，縁があってデータベース研究者として名を馳せた先生の研究室に入り，情報検索の研究活動を行っていたが... まさかデータベースの教科書を自分が書くとは思ってもいなかった．

目次

第 1 章　データベースを使わない世界 1
1.1　ケース 1: 販売履歴の記録をはじめる 1
1.2　ケース 2: サイトの認知度向上につき，得られるデータも膨大に!? 4
1.3　表計算ソフトでデータ管理を行ったときに起きる悲劇 6

第 2 章　データベースの基礎知識 9
2.1　データベースとは 9
2.2　大規模なデータを扱う上での課題 11
2.3　データベースを用いるメリット 14
2.4　関係データベース超入門 15

第 3 章　関係データモデル 21
3.1　関係データモデルのデータ構造 22
3.2　非正規関係と第 1 正規形 25
3.3　一貫性制約 26

参考文献 37

第 1 章

データベースを使わない世界

皆さんは「データベース」と聞いて何を思い浮かべますか．たくさんのデータの集まり —— こんなイメージを持つ人が多いのではないでしょうか．その直感は決して間違いではありません．

さて，情報系の学部やデータサイエンス学部の多くでは，「データベース」は必修科目として位置付けられています．ところで，データベースはわざわざ科目として立ててまで学ぶほど重要なトピックなのでしょうか．

結論から言えば，将来大規模なデータを扱う IT エンジニアやデータ分析者を目指す人であれば，学ぶ価値は「大いにあり」です．1960 年代後半から今日に至るまで，データベース技術は盛んに研究開発が行われてきました [1, 2]．なぜなら，大きなデータの集まりを適切に扱うには，対処しなければならない問題が数多く存在するからです．

本書では，大きなデータの管理や分析に興味がある初学者に向けて，データベースの仕組みを解説します．ただし，この第 1 章ではデータベースの話はいったん横に置いておきましょう．まずは，「扱うデータが増えるとどんな問題が起きるのか」，例を見ながら一緒に考えてみましょう．

1.1　ケース 1: 販売履歴の記録をはじめる

以下は，山畑さんという架空の人物のお話です．

山畑さんは家族で小さな小売店を営んでいる．個人経営ながら山畑さんのお店は繁盛している．とはいえ，街には大手チェーン小売店が進出してきており，このまま順調に経営を続けられるか，不安が募っている．何か手を打たなければならない．

2020 年の 4 月，山畑さんは念願のショッピングサイトを立ち上げた．ショッピングサイトを立ち上げたのは，オンラインの場にも顧客獲得の機会を求めるためだ．サイトは順調に立ち上がり，注文もポツポツ入ってきている．

ところで，最近「データサイエンス」なるものが世間の注目を集めているらしい．データを活かせばビジネスチャンスが広がるとのことだ．山畑さんは，Excel シートに記録を取り始めた販売履歴を分析してみようと思い立った．

山畑さんが使っている Excel シートには，「いつ，誰が，何を，いくらで購入したか」の情報が記録されている．ショッピングサイトは立ち上がったばかりであり，Excel シートには 200 行しかデータが入っていない．しかし，今後データが貯まっていけば，売り上げを増やすための課題が見えるかもしれない．いずれがっつりとデータ分析をやるためにも，山畑さんは手持ちのデータを用いて分析の練習に取り組むことにした．

さあ，皆さんも山畑さんになりきったつもりでデータ分析に取り組んでみましょう．こちらの URL（`https://dbnote.hontolab.org/data/purchase_small.xlsx`）には，山畑さんが分析しようとしている Excel ファイルがあります．まずは URL から Excel ファイル（`purchase_small.xlsx`）をダウンロードし，中身を確認してみてください[*1]．

図 1.1 は，山畑さんが分析の練習材料として用いた Excel データの一部です．図を見るとわかるように，この Excel シートでは以下の 5 項目についてデータが記録されています．

- `purchased_at`：購買（販売）日時
- `customer`：商品を購入した人物の氏名
- `gender`：商品を購入した人物の性別
- `product`：購入された商品名

[*1] データはすべて架空のものです．

	A	B	C	D	E
1	purchased_at	customer	gender	product	sale
2	2020-04-01	山本 裕美子	F	お茶飲料	160
3	2020-04-01	渡辺 真綾	F	果汁飲料	120
4	2020-04-01	高橋 くみ子	F	油菓子	200
5	2020-04-01	橋本 零	M	タバコ	1780
6	2020-04-01	渡辺 太一	M	浴室用品	530
7	2020-04-01	佐藤 和也	M	果汁飲料	120
8	2020-04-01	木村 七夏	F	機能性飲料ドリンク	240
9	2020-04-01	前田 舞	F	タバコ	1780
10	2020-04-01	鈴木 稔	M	チーズ	250
11	2020-04-01	林 太一	M	即席スープ	200
12	2020-04-01	近藤 結衣	F	紙オムツ	1930
13	2020-04-02	藤井 香織	F	蒲鉾	310
14	2020-04-02	吉田 千代	F	コンニャク	150
15	2020-04-02	渡辺 直子	F	調理補助器具キッチンシール	580
16	2020-04-02	渡辺 桃子	F	弁当	330
17	2020-04-02	中村 加奈	F	即席パスタ	220
18	2020-04-02	中村 あすか	F	プレミアムアイス	270
19	2020-04-03	加藤 淳	M	歯磨き	350
20	2020-04-03	渡辺 翼	M	畜産珍味	100

図 1.1: 山畑さんが分析に用いた初期の Excel データの一部.

- `sale`: 販売価格

ご存じの通り，Excel はビジネス現場で非常によく使われている表計算ソフトです．そのすべてを理解している人は少ないですが，データの絞り込みや集計など，便利な機能がたくさん備わっています．ここでは，山畑さんになりきって以下のクイズに取り組んでみてください．どのクイズも，販売履歴データの分析においてはよくあるタスクです．

Q1-購買頻度：あの人は何回買い物をしている？

Excel の検索機能を用いて，「岡田 真綾」という人物が何回買い物をしていたかを数えてください．

Q2-購入者の絞り込み：商品 X を購入しているのは誰？

Excel のオートフィルタ機能を用いて，「ビタミン補助剤」を購入している人をリストアップしてください．

Q3-総売上金額

Excel 関数の `SUM` を用いて，現時点での総売上金額を計算してください．

Q4-人気商品．最も売れた商品は？

Excel の「ピボットテーブル」機能を使って，集計期間中に

- 最も購買回数が多かった商品
- 最も売上金額の合計が大きかった商品

をそれぞれ求めてください．

いかがでしたでしょうか．Excel の基本的な機能を使えば，どのクイズもサクっと解けたのではないでしょうか．

Excel はビジネス上計算に必要な機能が一通り備わっています．なにより，大抵の PC にはインストールされているオフィスツールですから，手軽に使えるのが魅力です．学術研究の世界でも，データの簡単な集計や可視化にはしばしば Excel が使われます．

何も問題がないように見える Excel ですが，扱うデータの規模や用途が大きくなると，様々な問題に遭遇します．山畑さんのその後の話を例に，その問題点を一緒に考えてみましょう．

1.2 ケース 2: サイトの認知度向上につき，得られるデータも膨大に!?

現時点では手持ちのデータは少ないものの，販売履歴データの分析に将来性を感じた山畑さん．販売履歴データを有効活用できるよう，ショッピングサイト運営により力を入れる決意を固めたのでした．

以下は，山畑さんのその後の話（架空の話）です

ショッピングサイト立ち上げ以降，順調に利用者数も増えていった．やはりメディアに取り上げられたのが大きかったのだろう．あのタイミングでサイトの認知度が一気に高まり，サイトの利用者数や利用頻度も加速度的に増えていった．それに伴い，サイト運営に関わるスタッフも増員した．

販売履歴の管理は，当初は山畑さんが一人で担当していたが，さすがに一人

では対応しきれなくなった．そこで，ある時点から数名体制で販売履歴の記録を行うことになった．これまで販売履歴の管理に使ってきた Excel シートをクラウドストレージに置き，記録担当スタッフの PC 間で同期を取る仕組みを導入．同じ Excel ファイルの上で，スタッフ全員で販売履歴を記録できるようにしたのである．

2 年後．サイト事業は軌道に乗った．十分な量の販売履歴データが蓄積されたと判断した山畑さんは，いよいよ大規模な販売履歴データの分析に取りかかることを決意した．立ち上げ当初は 200〜300 行しかなかった Excel シートであったが，シートを開きその行数を数えてみると・・・なんとその数 90 万行以上！データの量に小躍りした山畑さんは，Excel シートの扱いに詳しいスタッフと共に，意気揚々とデータ分析に取りかかったのであった．

オンラインショッピング事業に成功した山畑さん，蓄積された販売データも膨大な量になりましたね．90 万行以上もある Excel シートは，まさに「巨大なデータのかたまり」になっています．

さあ，ここからは再度，山畑さんになりきってこの Excel シートを用いたデータ分析に取り組んでみましょう．こちらの URL（`https://dbnote.hontolab.org/data/purchase_large.xlsx`）には．上記ケース 2 で山畑さんが分析しようとしている Excel ファイルがあります[*2]．まずは，その Excel ファイル（`purchase_large.xlsx`）をダウンロードし，ファイルを開いてください．この Excel ファイルのシートは，先ほどのケース 1 で用いた Excel シートと同じ構造（項目）となっています．異なるのは，データの量（行数）だけです．もし，ファイルがうまく開けない場合は，こちらの URL（`https://dbnote.hontolab.org/data/purchase_medium.xlsx`）からダウンロードできる Excel ファイル（`purchase_medium.xlsx`）を用いてください．ダウンロードが完了したら，ケース 1 と同じタスク（Q1〜Q4）に取り組んでみてください．同じタスクでもデータが増えたことで異なる結果が得られるでしょうし，別の体験が得られるはずです．

[*2] 繰り返しになりますが，データはすべて架空のものです．

1.3 表計算ソフトでデータ管理を行ったときに起きる悲劇

ケース 1 および 2 で用いた Excel ファイルは，販売履歴データの集まりでした．一般的には，このようなデータの集まりは「データベース」ということになるでしょう．

ところで，ケース 2 のタスクに取り組んでみて，イライラを感じたりしませんでしたか．実はケース 2 の Excel ファイルには，大量のデータを管理・処理する上での課題を感じていただくために，いくつかの問題を仕込んでおきました．

ファイルを開いた方がまず感じたのは，Excel の動作が重い，ということでしょうか．Excel は，基本的にデータをすべてメモリに読み込もうとします．PC のメモリは作業用机のようなものです．メモリという机が広いほどたくさんの資料（データ）を開いて作業ができますが，机の広さを超えるデータ量を扱おうとすると，PC は身動きが取れなくなってしまいます．ケース 2 で扱った Excel シートは一般的な利用では扱わない量のデータが含まれている，スペック（メモリ）がそれほど高くない PC を使っている場合，Q1 から Q4 のタスクに取り組むどころか，Excel ファイルを開くことも難しいかもしれません．

無事 Excel ファイルを開くことができたとしても，ケース 2 の Excel ファイルのデータを分析するには骨が折れます．まずケース 1 に比べ，行数が圧倒的に多いです．そのため，Q1 タスクでは，Excel の検索機能を使って「岡田 真綾」さんの行を検索するにしても，データが多すぎて何度も「Enter キー」を押す羽目になります（しかも何回押したか分からなくなる）．また，ケース 2 の Excel データにはデータの誤記や不適切なフォーマットが混入しています．たとえば，`sale` の列に半角数字と全角数字が混在しています[*3]．`purchased_at` 列の日付は，「年月日」というフォーマットのデータもあれば，「年-月-日」というフォーマットのデータが混在しています．そのため，Q3 タスクでは，売り上げ金額の正しい合計値が得られません（`SUM` 関数は全角数字を無視する）．さらに，`product` 列では「セル（マス目）1 つにつき商品 1 つ」となっておらず，ある顧客が同時に複数の商品を購入した場合，1 つのセルにカンマ区切りで複数の商品が登録されてしまっています．

[*3] 一般に，計算機は半角文字（例: 123，ABC）と全角文字（例: １２３，ＡＢＣ）を区別します．

そのため，Q4 タスクではピボットテーブルを使っても，商品ごとの集計が行えなくなっています．

なぜ，このような問題が起きたのでしょうか．それは，本来 Excel は個人用の表計算アプリケーションであって，大規模データの管理や処理を前提として設計されていないためです．ビジネスの現場で扱うデータは，個人で扱うそれと比べて規模が大きく，多様になることが多いです．また，複数人，複数部署，複数サービスでデータを共有することも珍しくありません．そのため，誰かが編集中のデータを別の人が意図せず上書きをしようとしたり，データのフォーマット（形式）や事前に定めた入力ルールを無視してデータを入力してしまう，といった問題も起きやすくなります．このように，数十万行，数百万行ある表データを Excel で管理・処理しようとすると，さまざまな不都合が生じます（図 1.2）[*4]．

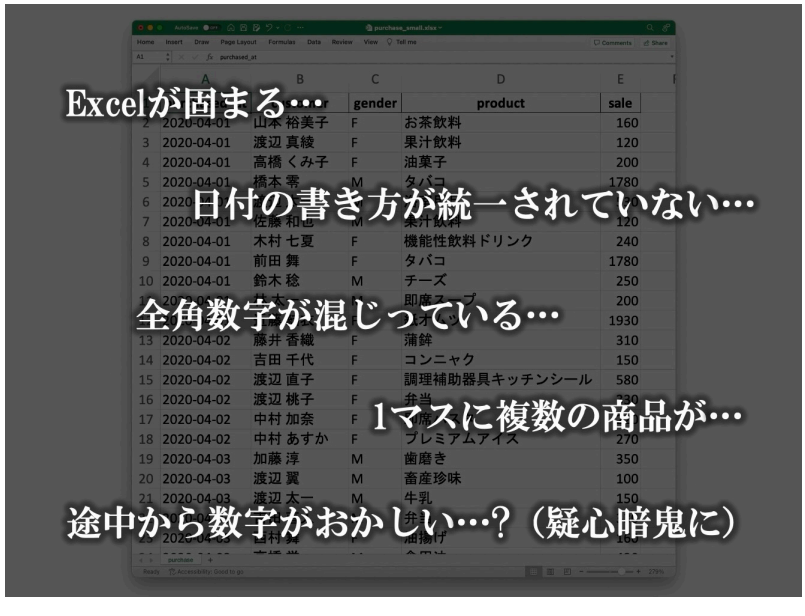

図 1.2: 大きな表データを複数人で Excel で扱うときの悲劇.

今日，企業や組織が扱うデータの量はますます増加しています．また，蓄積された大規模なデータを分析して，ビジネスに活かそうとする動きも活発化していま

[*4] 数十万，数百万行の表データは Excel で扱うには大規模ですが，本書で学ぶデータベースシステムにとっては小規模です．現代のデータベースでは，何千万，何億行という規模の表データを扱うことは珍しくありません．

す．では，大規模なデータをうまく管理・処理するためにはどうすればよいでしょうか．そのための技術こそが「データベース」です．「データベース」を用いれば，不適切なデータが混入することを防いだり，大量のデータを効率よく検索・集計することが可能になります [2]．以降，本書では**大規模データを効率よく管理・処理するための「データベース」技術** について解説します．

第 2 章

データベースの基礎知識

今日，データベースは様々な業務やアプリケーションで利用されています．例えば，Amazon[*1]や楽天市場[*2]といったオンラインショッピングサイトにおいては，購買履歴管理や在庫管理を行うためにデータベースが用いられています．Instagram[*3]や Tiktok[*4]といった，多数のユーザがコンテンツを投稿・閲覧するソーシャルメディアにおいてもデータベースは欠かせません．個人用ブログのような比較的小規模なウェブサイトでも，記事データを管理するためにデータベースが用いられています（例: WordPress[*5]．このように，データを語る上でデータベースはなくてはならないものです．

本章では，データベースの概念を説明した後，最も代表的なデータベースであり，かつ本書の主要テーマでもある**関係データベース**の概要について述べます．

2.1 データベースとは

データを収集・利用する文化が根付いてきたこともあり，データベースという言葉が市民権を得つつあります．しかし，「データベース」という語が指す意味は，一般人と IT の専門家とは大きく異なります．第 1 章でも述べたように，一般の方にとってのデータベースとは「データの集まり」くらいの意味で捉えられていること

[*1] Amazon. `https://www.amazon.co.jp/`
[*2] 楽天市場. `https://www.rakuten.co.jp/`
[*3] Instagram. `https://www.instagram.com/`
[*4] Tiktok. `https://www.tiktok.com/`
[*5] WordPress. `https://ja.wordpress.org/`

が多いです[*6]．一方，IT の専門家にとってのデータベースは，

> “複数の応用目的での共有を意図して，組織的にかつ永続的に格納されたデータ群（北川博之著「データベースシステム」[3] より）”

あるいは

> “データの正しさを管理する主体によって体系的に整理され，計算機によって永続的に格納されたデータの集まり（吉川正俊著「データベースの基礎」[4] より）”

を意味します．「データの正しさを組織的に管理する」という考えが強調されています．

データベースを扱うためのシステムは，**データベース管理システム（database management system, DBMS）**と呼ばれます．我が国のデータベース研究の大家である増永良文氏はその著者の中で，データベースと DBMS の関係を「料理」と「それを盛る器（うつわ）」の関係に例えています [5]．

- データの正しさを組織的に管理するための「器」が DBMS
- DBMS という器に盛られた「料理」，すなわち正しく管理されたデータの集まりがデータベース

という対応関係になります．本来データベースとは DBMS によって管理されるデータ群を指しますが，データ管理の現場では，DBMS そのもの（あるいは DBMS とそれによって管理されるデータ群）をデータベース（DB）と呼ぶこともあります．

増え続ける多様かつ大量のデータと付き合っていくためには，データのうまい管理・処理方法が必要になります．場当たり的にデータを作ってはファイルとしてフォルダ（ディレクトリ）に突っ込んでおいたり，Excel シートにまとめていくというやり方は，扱うデータが増え，データを使う人間やアプリケーションが増えるにつれて，やがて破綻します．IT の専門家が考えるデータベースは，こういった事態を未然に防ぐための強力なツールとなります．

[*6] データと情報も明確に区別されることなく使われることが多いですが，情報学の世界では区別されています．データは「様々な活動や営為の結果得られる記号の集まり」であり，情報は「データが解釈され，意味や価値が付加されたもの」を指します．

2.2　大規模なデータを扱う上での課題

第 1 章の山畑さんの事例で見たように，大量のデータを適切かつ効率的に管理・処理するには，以下のような要件が求められます．

多様かつ大規模なデータの管理

ビジネスをはじめとする現場では，多種多様かつ大量のデータが時々刻々と発生しています．例えば，2020 年 1 月 1 日から 2020 年 12 月 31 日までの 12 か月間に，Amazon.co.jp で購買された商品の数は 5 億点以上にのぼるとされています [6]．

個人で管理する家計簿のような小規模な表データであれば，Excel のような表計算ソフトでも対応できます．しかし，データの規模が大きくなり，様々な種類のデータが登録されるようになり，表データの登録・更新に関わる人が増えていくと…やがてデータの管理が破綻するのは想像に難くありません．何がどこに登録されているのかもよく分からなくなりますから，目当てのデータを探すのも大変です．データの登録，修正，削除などなど，何をするのも大変になります．

そもそも Excel は 1 つの表につき最大 100 万行程度しか扱えません．Excel のようなスプレッドシートでは，購買データのように数千万行，数億行もある多様かつ大規模なデータを効率よく蓄積，管理することは困難極まりないのです．

データの正しさの保証

管理するデータが大量かつ多様になってくると，データを正しく保つことが難しくなってきます．特に，大量のデータを複数人，複数のグループや部署で扱うようになると，データの入力ミス，データ入力規則の無視，データ間の矛盾が発生しやすくなります．

当然ですが，管理対象となるデータに誤りが混入すると，そのデータを使うときに大変なことになります．例えば，図 2.1 のように，購買データの中に

- 現実にはあり得ない売り上げ金額が記録されている（データの入力ミス）
- 入力形式が異なる日付が混在している（データの入力規則の無視）
- 取扱商品リストにないはずの商品が購買履歴に含まれている（データ間の

矛盾）

といったことが起きると，オンラインショッピング事業においては一大事です．このような正しくないデータは直接参照されたときも問題になりますが，複数のデータを集計する際にも悪影響を及ぼします．それゆえ，データ管理においては，格納されるデータの正しさ，データ間の矛盾のなさを保証する機能が求められます．

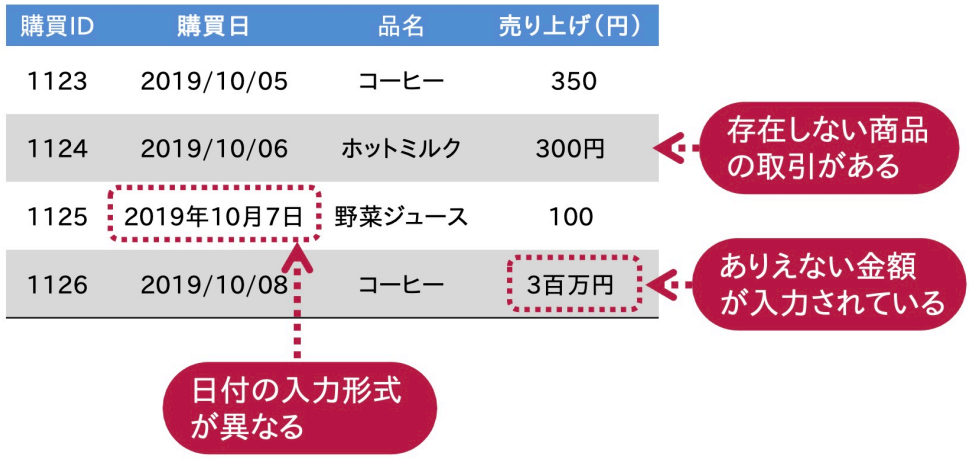

図 2.1: 正しくないデータの例.

高速で効率的なデータ処理

大量のデータを正しく管理できたとしても，対象となるデータを高速に処理できなければ実用的ではありません．例えば，図書館における書籍の検索システムを考えてみよう．たとえ数百万件の書籍情報を正しく管理する蔵書管理システムがあっても，ニーズにマッチする書籍のリストの検索に数分待たされるようでは，誰も使いたくありません．

一般に，あるシステムを利用するユーザが増えるにつれて，システムにかかる負荷も増えていきます．例えば，Amazon.co.jp では毎分平均 900 件以上の商品取引が行われています [7]．それゆえ，大量のデータが格納されているシステムにおいては，多発的に大量のデータのやりとりが発生することも想定して，高速かつ効率的なデータ処理が求められます（図 2.2）．

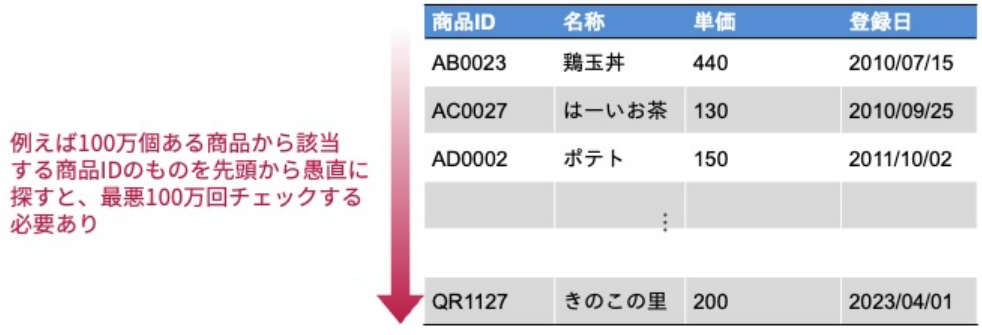

図 2.2: 上から下に順に調べ上げていく線形探索では．大規模データに対する高速なアクセスは難しい．

同時実行（並列処理）

関連して，大勢の人が同時にデータを追加，更新，削除しても，管理中のデータに矛盾が生じないようにシステムが動作することも重要です．

例えば，X さん，A さん，B さんが PoyPoy という名称のコード決済システムを利用しているとしましょう．最近のコード決済システムはお店にお金を支払うだけでなく，ユーザ同士でお金を送金しあうこともできます．さて，X さんの PoyPoy 口座残高は 5 万円とします．一般に，口座への送金があった際，送金先口座の残高の計算は

残高 = 送金時の「送金先口座」の残高 + 送金額

送金元口座の残高の計算は

残高 = 送金時の「送金元口座」の残高 - 送金額

となります．これは至極当然の処理ですが，次のような状況を考えてみましょう．

ある日，X さん，A さん，B さんは食事に出かけ，X さんが A さん，B さんの代金を立て替えたとします．A さんと B さんは，PoyPoy アプリを通じて X さんに立て替えてもらった金額（3000 円）を精算することにしました．さて，A さんと B さんは，X さんの PoyPoy 口座に**まったく同じタイミングで** 3000 円送金したとします．この際，前述の計算式をそのまま適用すると，図 2.3 のように誤った処理

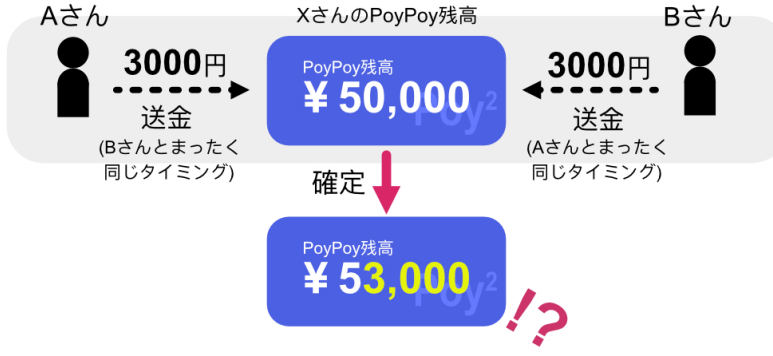

図 2.3: 複数の処理要求を矛盾なく処理できていない例.

が行われてしまいます．A さんと B さんが送金を行った後，X さんの口座の残高は 5 万 3000 円になりました – こんなことが起きたら大変ですよね．この例における問題点は，たとえ振込み処理が同時に発生したとしても，A さんの振り込み処理を待ってから B さんの処理を行うべきであったという点です．

このように，データ処理の内容によっては，別の処理が完了したことを保証してから次の処理を行う必要があります．さらに，処理の途中で何らかのエラーが起きた場合は，すべての処理をキャンセルして最初の状態に戻すことが求められます．データを管理しているシステムに同時アクセスがあった場合でも，データを矛盾なく処理できることが重要となります．

アクセス権限のコントロール

データによっては，誰でも自由に閲覧してよいものもあれば，特定の立場のユーザしかアクセスできないようにすべきものも存在します．多種多様なデータのやりとりが発生する環境においては，ユーザの属性ごとにデータの閲覧，作成，更新，削除といったアクセス権限を制御する機能が必要となります．

2.3　データベースを用いるメリット

一人で扱えるほどデータの規模が小さければ，Excel などの表計算用のソフトウェアでデータ管理しても問題はありません．しかし，扱うデータが多様かつ大規模になると，問題が生じなくするために要求されるデータ処理の質が変わります．

そのため，データの管理方法や処理方法を見直す必要があります．

本稿で学ぶデータベースを用いれば，**データの一元管理**が可能となり，データの管理や利活用において様々な恩恵を受けられます．具体的には，データベースが備える以下のような特性あるいは関連技術を用いることで，前節で述べた「大規模なデータを扱う上での課題」をクリアすることができます．

- 大規模なデータの管理：「**物理的データ格納方式**」の工夫によって対応
- データの正しさの保証：「**一貫性制約**」によって対応
- 高速で効率的なデータ処理：「**索引づけ**」「**問い合わせ最適化**」などで対応
- 同時実行，データの共同利用：「**トランザクション**」などで対応
- アクセス権のコントロール：「**ロール管理**」によって対応

2.4　関係データベース超入門

これから本書で学ぶ**関係データベース（relational database）**とは，あらゆるデータを**表（table）**[*7]の形式で管理するデータベースです [8]．第 3 章で述べるように，実はデータベースといっても様々な種類のものがありますが，関係データベースはその中でも最も代表的なものです．関係データベースを管理するためのシステム（すなわち DBMS）は，**関係データベース管理システム（relational database management system，略して RDBMS）**と呼ばれています．

図 2.4 は，授業の履修状況・成績を管理する関係データベースの例です．この関係データベースには，以下の 3 つの表が存在しています．

- 学生テーブル：学籍番号，氏名，入学年度，所属といった学生に関する情報を格納
- 科目テーブル：科目 ID，科目名，開講年度といった科目に関する情報を格納
- 履修テーブル：どの学生が何の科目を履修し，どのような成績であったかに関する情報を格納

各表は行と列から構成されており，各行には表が扱うデータが格納されています．図 2.4 の学生テーブルを例に取ると，各行のデータは学生一人一人の情報を表し

[*7] 表のことをテーブルと呼ぶこともある．

学生

学生ID	姓	名	入学年度	所属
s00001	川澄	桜	2023	A学部
s00002	山畑	滝子	2024	B学部
s00003	田辺	通	2024	C学部
		⋮		

科目

科目ID	科目名	開講年度
c0001	線形代数	2023
c0002	線形代数	2024
c0003	統計学入門	2024
	⋮	

履修

科目ID	学生ID	成績
c0001	s00001	不可
c0002	s00001	良
c0002	s00002	優
c0003	s00003	可
	⋮	

図 2.4: 関係データベースの例.

ています．関係データベースでは，行のことを**タプル（tuple）**あるいは**レコード（record）**と呼びます．各レコードは，表の見出しに対応するデータの値を持ちます．各見出しのことを**属性（attribute）**，属性に対応するレコードの値を**属性値（attribute value）**と呼びます．学生テーブルの例では，学生 ID，姓，名，入学年度，所属が属性に対応し，一番上に書かれているレコードの属性「学生 ID」の属性値は `s00001` となっています．

この例だけ見ると「なんだ，関係データベースは単なる表じゃないか」と思われたかもしれませんが，**ただの表ではありません**．一見するとただの表にしか見えませんが，データを正しくかつ効率よく管理・処理するために，あらかじめ定義された「データの構造」「データの制約」「データの操作」に関する規則に従ってデータが作られ，関係データベース内に格納されます[*8]．例えば図 2.4 の関係データベースであれば，以下のような規則に従い，授業の履修状況や成績を管理するための情

[*8] データの構造，制約，操作に関する規則を定めたものを「データモデル」と呼びます．データモデルについては，第 3 章で詳しく述べます．

報が複数のテーブルに分割され，データベース内に格納されています：

- 学生テーブルは「学生 ID」「姓」「名」「入学年度」「所属」という属性をもつ
- 学生テーブルの「学生 ID」や科目テーブルの「科目 ID」はテーブル内で値が重複することはない
- 履修テーブルには，科目名や学生の氏名といった科目や学生の詳細情報は格納しない
- 履修テーブルの成績には「優」「良」「可」「不可」のいずれかしか登録できない
- 履修テーブルに現れる科目 ID および学生 ID は，必ず学生テーブルと科目テーブルに存在する

少し詳しく見てみましょう．図 2.5 のように，学生テーブルの「学生 ID」は重複することがないため，その値が決まれば学生の情報が一意に決まります（どの学生かが特定できる）．このような属性（またはその組み合わせ）テーバーン委データベースでは**主キー（primary key）**と呼びます．

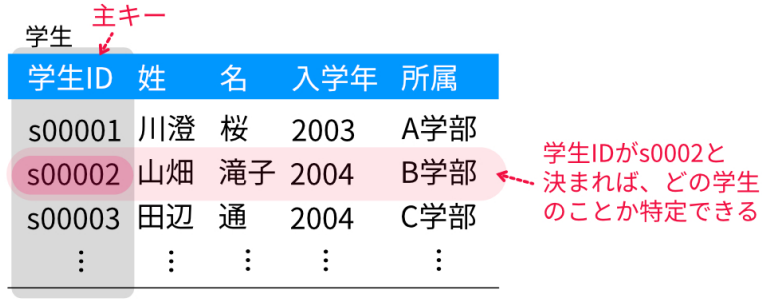

図 2.5: 主キーの役割

履修テーブルはどの学生が何の科目を履修し，どのような成績であったかを管理するための表ですが，学生の具体的な情報は保持せず，学生 ID のみを保持しています．学生の姓や名，所属といった情報が欲しい場合は，学生 ID を手がかりに学生テーブルを参照すれば取得できます．同じことは科目情報についても言えます．これは見方を変えると，履修テーブルに格納された学生 ID や科目 ID は，必ず学生や科目の詳細情報を参照できるよう，必ず学生テーブルや科目テーブル内に存在している必要がある，ということになります．このように，他のテーブルでは主キー

となっていて，それを手がかりにすることで関連情報を参照するための属性を，関係データベースでは**外部キー（foreign key）**と呼びます．図 2.6 の例では，履修テーブルにおける学生 ID や科目 ID は（学生テーブルや科目テーブルに対する）外部キーとなっています．（直感的なイメージとしては）外部キーは，ウェブページにおけるハイパーリンクのような役割を果たしています．先の例で言えば，履修テーブルの学生 ID をクリックすると，学生テーブルから該当学生の詳しい情報が見れる，といったイメージです．このような仕組みによって，関係データベースは情報が冗長にならないように整理整頓されています．

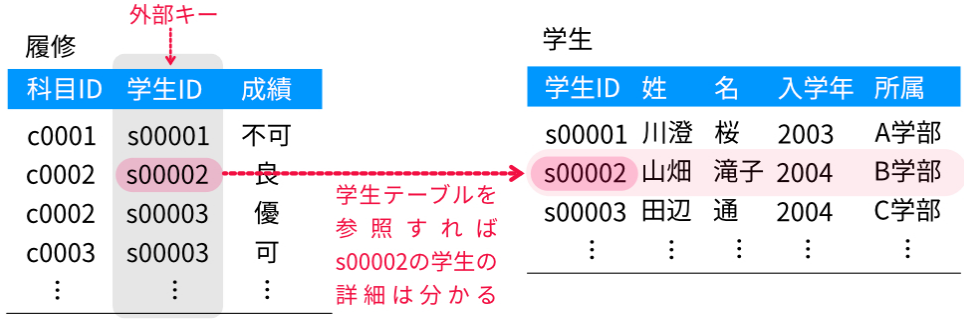

図 2.6: 外部キーの役割

なぜ履修テーブルに学生や科目の詳細情報も載せないのでしょうか．それは，学生や科目の詳細情報が変更された場合に，履修テーブル内の情報もすべて更新しなければならなくなるからです．もしすべての情報を 1 つの表にまとめていたら，どうなるでしょうか．例えば，山畑さんが在学中に B 学部から D 学部に転部し，所属が変わったとしましょう．この場合，もし履修テーブルに山畑さんに関する情報が 100 件登録されていたら，100 件すべての所属を B 学部から D 学部に書き換えなければなりません．その際，もし 1 件でも書き換え忘れがあったら，B 学部の山畑さんと D 学部の山畑さんが混在することになり，正しい成績証明書が出せなくなってしまいます．図のように，学生や科目の詳細情報を履修テーブルとは別のテーブルに分けておけば，学生や科目の詳細情報が変更された場合でも，学生テーブルや科目テーブル内の情報を 1 か所更新すれば済みます．このような工夫も，関係データベースの重要な特徴の 1 つです．

上記のような工夫はデータの正しさを保証するためのものでしたが，他にも関係

データベースにはデータの検索・集約を効率よく行うための仕組みや同時アクセスの制御など，データを適切に扱うための様々な仕組みが備えられています．次章以降では，

- 一見ただの表にしか見えない関係データベースが，どのような理論に基づいて定式化されているか
- 関係データベースをどのように設計するのか
- 関係データベースを用いると，なぜ大規模データの管理が効率的になるのか

などについて詳しく述べていきます．

クイズ

Q1. メジャーなデータベース管理システム

ウェブ検索エンジンを用いて，世の中にあるメジャーなデータベース管理システムを調べなさい．また，調べたデータベース管理システムを「関係データベースを扱うもの」と「そうでないもの」に分類しなさい．

Q2. 線形探索

100 万件ある商品リストの中に特定の商品が含まれているかを確認したいとします．商品リストの先頭から末尾まで順に商品名を確認していくと，平均で何回（何件）の確認で商品の有無を確認できるでしょうか．

Q3. データベース管理システムの利用例

普段利用しているサービスのうち，データベース管理システムを用いていると思われるものを 3 つピックアップしなさい．

Q4. CSV/TSV ファイル

CSV ファイルおよび TSV ファイルとは何かを調べなさい．また，（データベース管理システムでデータを管理する場合と比較して）CSV/TSV ファイルに存在する欠点を挙げなさい．

Q5. **チューリング賞**

関係データベースの理論的基盤である関係モデルを提唱した Edgar F. Codd 氏は，計算機科学分野のノーベル賞といわれるチューリング賞の受賞者である．Codd 氏以外で，データベースに関する功績でチューリング賞を受賞した人をピックアップしなさい．

第 3 章

関係データモデル

関係データベース（relational database） とは，関係データモデルにもとづき表現されたデータの集まりである．関係データモデルは，最も代表的なデータモデルである．1970 年に Edgar F. Codd 氏により提案されたもので，単純ながらその背後には強力な数学的基盤をもつ [8]．関係データモデルでは，あらゆるデータを**表（table）**としてモデル化する．一見すると単純な表にしか見えない関係データモデルは数学によって定式化されており，強固な理論的基盤を有している．関係データモデルを用いることで，データから冗長性を排除し，データを正しく管理することが可能となる．

以下では，数学的な観点から関係データモデルについて述べる．少々堅い説明にはなるが，関係データベースという計算機科学技術が数学に支えられていることを知るためにも，眠気を我慢して読んでほしい．なお，本講では数学における「集合と写像」の知識を使う．

Note - 計算機科学とは抽象化の学問

計算機科学（あるいは情報科学）というと「プログラミングでしょ」と考える学生は多いが，それは間違いである．計算機科学の本質は，**物事を計算可能な状態にする（抽象化）して，計算機の上で処理・分析**することにある．データベースもその一つである．計算機科学は抽象化の学問なのである．

表 3.1: ある小売店の商品に関するデータを収めた関係

商品

商品 ID	名称	単価	登録日
P1	はーいお茶	130	2020/07/15
P2	午前の紅茶	130	2020/09/25
P3	健康麦茶	150	2021/02/16
⋮	⋮	⋮	⋮
P1000	きのこの里	200	2023/01/08

3.1　関係データモデルのデータ構造

表 3.1 は，ある小売店で取り扱っている商品のデータを関係データモデルによって表現したものである．この表において，各行は小売店で取り扱っている商品に対応し，各列には各商品に関するデータが記述されている．

表の左上にある「商品」はこの表の名前を示しており，**関係名**と呼ばれる．関係データモデル上では，各行は**タプル（tuple）** と呼ばれる．表の 1 行目には，「商品」がどのようなデータを持つかを示す見出しが付けられている．表における各見出しに対応するものを，関係データモデルでは**属性（attribute）**と呼ぶ．また，各タプルにおけるある属性の値を**属性値**と呼ぶ．例えば，表中の 2 行目に対応するタプルは

$$(\text{“}P1\text{”}, \text{“はーいお茶”}, 130, \text{“2020/07/15”}) \tag{3.1}$$

であるが，このタプルの属性「単価」の属性値は `130` となる．

3.1.1　数学における「関係」

ところで，関係データベースあるいは関係データモデルの「関係」とは一体何だろうか．**関係（relation）** とは数学上の概念である．集合 $S_1, S_2, ..., S_n$ が与えられたとき，それらの直積集合 $S_1 \times ... \times S_n$ とは

$$S_1 \times ... \times S_n = \{(x_1, ..., x_n) \mid x_1 \in S_1, ..., x_n \in S_n\} \tag{3.2}$$

と表現されるものである．（数学上の概念である）関係とは，このような直積集合の部分集合を意味する．正確に書くと，集合 $S_1, S_2, ..., S_n$ が与えられたとき，直積集合 $S_1 \times ... \times S_n$ の部分集合を $S_1, S_2, ..., S_n$ 上の **n 項関係**と呼ぶ．

例えば，$A = \{2, 3\}$，$B = \{-2, -3\}$ が与えられたとき，直積 $A \times B$ は集合 A および B の要素のすべての組み合わせであるから

$$A \times B = \{(2, -2), (2, -3), (3, -2), (3, -3)\} \tag{3.3}$$

となる．このとき，その部分集合の 1 つである

$$\{(2, -3), (3, -2)\} \tag{3.4}$$

は集合 A および B 上の 2 項関係となる．このような（n 項）関係の概念を使って，関係データモデルは理論構築されている．

3.1.2　関係データモデルにおける「関係」

関係データモデルにおける関係は「数学における関係」を拡張したものになっており，以下の 2 つから構成される．

- 関係スキーマ
- インスタンス

関係スキーマ（relation schema） とは，関係の名前と属性の集合，および（後述する）一貫性制約の情報を示すものである．関係スキーマは

$$(\boldsymbol{R}(A_1, ..., A_n), \{\sigma_1, \sigma_2, ..., \sigma_m\}) \tag{3.5}$$

の形式で記述される．ここで

- $\boldsymbol{R}$ は関係名
- $A_1, ..., A_n$ は属性
- $\sigma_1, ..., \sigma_m$ は一貫性制約

に対応する．一貫性制約が自明な場合やそれを考慮しないときは，関係スキーマを

$$\boldsymbol{R}(A_1, ..., A_n) \tag{3.6}$$

のように簡潔に表記することもある．表 3.1 の関係「商品」の例の場合，関係スキーマは

$$\text{商品（商品}\,ID, \text{名称}, \text{単価}, \text{登録日）} \tag{3.7}$$

と記述する．

関係スキーマに記された各属性には，属性が取り得る値の集合を定義する．この集合は**ドメイン（domain; 定義域）** と呼ばれ，属性 A のドメインは $Dom(A)$ と記す．例えば，表 3.1 の関係「商品」の例では，属性「単価」は金額を表すので，そのドメイン Dom(単価) は 0 以上の整数値が想定される．これを数学的に記すと，以下のようになる:

$$Dom(\text{単価}) = \{x \mid x \in \mathbb{N}\} \tag{3.8}$$

同様に，属性「商品 ID」「名称」「登録日」についても，以下のようにドメインを定義できる．

$$Dom(\text{商品}\,ID) = \{x \mid x \in P\ \text{から始まる文字列集合}\ \} \tag{3.9}$$

$$Dom(\text{名称}) = \{x \mid x \in \text{文字列集合}\ \} \tag{3.10}$$

$$Dom(\text{登録日}) = \{x \mid x \in \text{年月日集合（ただし西暦表記）}\ \} \tag{3.11}$$

インスタンス（instance） は，関係スキーマ $\boldsymbol{R}(A_1, ..., A_n)$ におけるドメイン $Dom(A_1), ..., Dom(A_n)$ の直積の部分集合である．仮にインスタンスを R とすると，

$$R \subseteq Dom(A_1) \times ... \times Dom(A_n) \tag{3.12}$$

である．インスタンスは，前節で説明した数学上の関係に相当する．また，インスタンス R の要素がタプルとなる．

例に戻って，定義と照らし合わせてみよう．表 3.1 の関係「商品」の関係スキーマは 商品 (商品 ID, 名称, 単価, 登録日) であった．各属性のドメインの定義も与えたが，その定義から，例えば属性「単価」については

$$Dom(\text{単価}) = \{1, 2, 3, ...10000, ...\} \tag{3.13}$$

のようにあらゆる自然数を取り得る．また，属性「名称」については

$$Dom(\text{名称}) = \{\text{“あ”}, \text{“い”}, ..., \text{“はーいお茶”}, ...\} \tag{3.14}$$

のようにあらゆる文字列を取り得る．

各属性の直積 $Dom(\text{商品}\,ID) \times Dom(\text{名称}) \times Dom(\text{単価}) \times Dom(\text{登録日})$ には，

$$(\text{“}P001\text{”}, \text{“こんな商品はありません”}, 100\text{ 兆}, \text{“2020/01/01”}) \tag{3.15}$$

のように，実際に商品情報としてありえないタプルも含めて，様々なタプルが要素として含まれる．なぜなら，直積とは集合（ドメイン）の要素のすべての組み合わせであるためである．ドメインの直積の部分集合をインスタンスとすることは，取り得るタプルの候補から実際にデータとして扱われるタプルの集合を選択することに相当する．

Note - タプルは重複が許されない

一般的に使われる表では，同じ内容を表す行が複数あっても許される．一方，関係データモデルに基づく表は行，すなわちタプルの重複は許されない．なぜなら，関係データモデルのインスタンスは（数学的な）集合だからである．

3.2　非正規関係と第 1 正規形

関係データモデルにおけるドメインとは属性が取り得る値の集合を意味するが，ドメインの要素はそれ以上分解不可能な値を意味する**原子値（atomic value）**であることを想定している．原子値の例としては，文字列，整数，実数，真偽値，日付などが挙げられ，どれもデータの基本単位と呼べるものである．逆に原子値で「ない」例としては，タプルや集合，関係などが挙げられる．ドメインとして原子値以外の値をとることを許す関係を**非正規関係（unnormalized relation）**と呼ぶ．逆に，ドメインとして原子値しかとらない関係は**第 1 正規形（first normal form; 1NF）である**という．

例えば，図 3.1 の (a)(b) はともに商品の購買履歴を表しているが，(a) は属性「購入品目」に集合要素が入ってしまっている（例えば，`{"はーいお茶", "きのこの里", "のど飴"}`）．(a) の属性「購入品目」のドメインが原子値以外の値（つまり文字列の集合）を許してしまっていることから，(a) は非正規関係と見なせる．

一方，図 3.1(b) は (a) の購買品目の値が原子値（文字列の集合ではなく文字列）になるように設計されているため，第 1 正規形と見なせる．図 3.1(a) では 1 行で

表(a): 購買履歴

購買ID	購買品目	売上	登録日
T1	はーいお茶, きのこの里, のど飴	530	2020/07/15
T2	午前の紅茶	130	2020/09/25
T3	麦茶, おにぎり	250	2021/02/16
	⋮		
T1000	きのこの里	200	2023/01/08

分解

表(b): 購買履歴

購買ID	購買品目	売上	登録日
T1	はーいお茶	130	2020/07/15
T1	きのこの里	230	2020/07/15
T1	のど飴	170	2020/07/15
T2	午前の紅茶	130	2020/09/25
T3	麦茶	120	2021/02/16
T3	おにぎり	130	2021/02/16
	⋮		
T1000	きのこの里	200	2023/01/08

図 3.1: 非正規関係

表現できていた 1 回分の購買データが，(b) では複数行を使って表現されている．図 3.1(b) のほうがデータ構造が簡潔になるため，データが扱いやすい．

3.3　一貫性制約

一貫性制約（integrity constraint） とは，データベースが対象とする実世界を反映するように設定された，データが満たすべき規則である．関係データモデルにおいては，代表的な一貫性制約として以下がある：

- ドメイン制約
- キー制約
- 参照制約
- データ従属性

関係データモデルを扱うデータベースを設計する際には，上記一貫性制約を踏まえて**関係スキーマの定義や（複数の関係スキーマへの）分解** を行う．

3.3.1　ドメイン制約

ドメイン制約（domain constraint） とは，関係 $\boldsymbol{R}(A_1, ..., A_n)$ に含まれるタプルの各成分は，対応する属性のドメインの要素でなければならないという制約である．これは属性の定義ですでに触れたことである．ドメイン制約では，各属性のドメインのデータ型（例: 整数，実数，文字列，日付）に加えて，値が取り得る範囲を指定することもある．例えば，

先の例でとりあげた関係

$$\text{商品}\ (\text{商品}\ ID, \text{名称}, \text{単価}, \text{登録日}) \tag{3.16}$$

において，属性「単価」や「登録日」のドメインは

$$Dom(\text{単価}) = \{x \mid x \in \mathbb{N}\} \tag{3.17}$$

$$Dom(\text{登録日}) = \{x \mid x \in \text{年月日集合（ただし西暦表記）}\} \tag{3.18}$$

のように定義されていた．これらはドメイン制約である．これら制約にもとづき，

- 「単価」属性の値は**必ず**自然数
- 「登録日」属性の値は**必ず**西暦表記の年月日集合

でなければならない．よって，表 3.2 に記した表「商品 (*)」の各タプルは関係「商品（商品 ID, 名称, 単価, 登録日）」のインスタンスには**なれない**．なぜなら，

- 商品 ID が P1 の商品の登録日は年月はあるが日が抜けている
- 商品 ID が P3 の商品の登録日は和暦で表現されている
- 商品 ID が P1000 の商品の単価は漢数字（文字列）で表現されている

からである．

3.3.2　キー制約

まず，キー制約の前提となるキーの概念について説明する．関係 $\boldsymbol{R}$ における**超キー（super key）** とは，関係 $\boldsymbol{R}$ における属性の集合のうち，それらの属性値が決まればおのずと関係 $\boldsymbol{R}$ のタプルが唯一ひとつに決まる（タプルを一意に特定できる）ものを指す．

表 3.2: ドメイン制約に違反する関係

商品 (*)

商品 ID	**名称**	**単価**	**登録日**
P1	はーいお茶	130	2020/07
P2	午前の紅茶	130	2020/09/25
P3	健康麦茶	150	令和 3 年 2 月 16 日
⋮	⋮	⋮	⋮
P1000	きのこの里	2 百	2023/01/08

例えば，表 3.3 は関係スキーマ

$$\text{連絡先}\ (\text{学生}\ ID, \text{名前}, \text{学年}, \text{大学}\ email, \text{自宅住所}) \tag{3.19}$$

に従うデータを表形式で記したもので，各行（タプル）は学生の連絡先情報を示している．大学において学生情報を管理する場合，学生 ID（の値）が決まれば唯一ひとつの学生連絡先情報（行; タプル）を照会できるよう，学生 ID は重複がないように設定される．それゆえ，この関係「連絡先」において属性「学生 ID」は超キーとなる．同様に，属性「大学 email」も超キーとなる．

なお，{ 学生 ID, 名前 }，{ 学生 ID, 名前, 学年 } のように学生 ID を含む属性の集合も超キーになる．例えば，「学生 ID」が `S1`，「名前」が **川澄桜** である行は唯一 1 行に決まるので，属性集合 { 学生 ID, 名前 } も超キーになる．これは属性「学生 ID」が超キーであるから，当たり前である．

一方，属性「住所」は超キーにはなり得ない．学生 ID が `S3` と `S4` の学生の住所が同じ（つまり同じところに住んでいる）であるから，住所を 1 つ指定しても連絡先タプルが一意に特定できないからである．

超キーは複数ありえるが，上の例では属性「学生 ID」のみでタプルを特定できるように，属性集合　学生 ID, 名前　を使ってタプルを特定しようとするのは無駄であろう．そこで，超キーのうち極小（つまり最も小さい部分集合）のものを**候補キー（candidate key）** と定義する．候補キーは単純に**キー（key）** と呼ばれることもある．例えば，上の関係「連絡先」では，属性「学生 ID」や「大学 email」が候補キーとなりえる．

表 3.3: 関係「連絡先」

連絡先

学生 ID	**名前**	**学年**	**大学 email**	**自宅住所**
S1	川澄 桜	4	kawasumi@xxx.ac.jp	名古屋市 xxx
S2	山畑 滝子	3	taki@xxx.ac.jp	岡崎市 xxx
S3	田辺 通	3	t.tanabe@xxx.ac.jp	尾張旭市 xxx
S4	田辺 瑞穂	1	m.tanabe@xxx.ac.jp	尾張旭市 xxx
⋮	⋮	⋮	⋮	⋮
S1000	北 千種	2	kita@xxx.ac.jp	名古屋市 xxx

最後に**主キー（primary key）** の概念を導入する．候補キーのうち，値として**未定義や空の値（空値または NULL 値と呼ばれる）** を取る可能性がなく，かつデータベース管理上都合のよいキーの 1 つを主キーと決める．主キー以外の候補キーは**代替キー**と呼ばれる．例えば，先の関係「連絡先」の場合，候補キーは「学生 ID」と「大学 email」であったが，大学 email は「氏名 xxx.ac.jp」のようなものから「学籍番号 xxx.ac.jp」のようなものに変更される可能性があったとしても，学生 ID は一度決めたらほぼ変わらないと思われる．これらを鑑みると，関係「連絡先」においては属性「学生 ID」を主キーとして選ぶのがよさそうである．

なお，候補キーの定義から主キーは属性の集合をとることができる．このような主キーを**複合主キー（composite primary key）** と呼ぶ．例えば，表 3.4 は，学生が履修した科目の成績をあらわす関係「履修」を表にしたものである．この表においては科目 ID あるいは学生 ID だけでは成績が特定できないが，科目 ID と学生 ID の両方が決まれば，ある学生のある科目の成績が特定される．それゆえ，関係「履修」においては { 科目 ID, 学生 ID } の属性ペアが主キーとなる．

関係表と主キー，候補キー，超キーの関係は，図 3.2 のようにまとめることができる．

ここまで，超キー，候補キー，主キーの説明を行ったが，最後に本節の主題であるキー制約について述べる．**キー制約（key constraint）** は，関係 $\boldsymbol{R}$ の関係スキーマに対して主キーが設定されたとき，$\boldsymbol{R}$ のインスタンスにおいて

表 3.4: 関係「履修」

履修

科目 ID	学生 ID	成績
C1	S1	不可
C2	S1	良
C2	S2	優
C3	S3	可
⋮	⋮	⋮

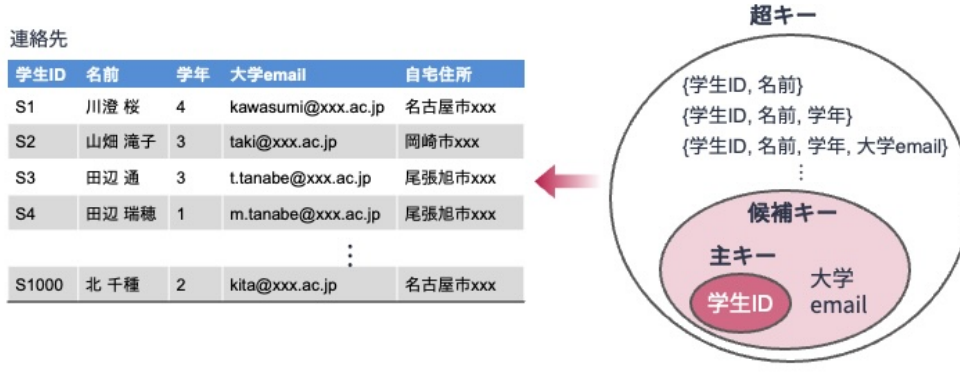

図 3.2: キーの概念の整理

- 主キーに設定された属性の値は重複があってはならない，かつ
- その要素（タプル）は主キーによって一意に特定されなければならない，かつ
- 主キーとなる属性の値は **NULL 値** であってはならない

という制約である．定義は小難しく見えるかもしれないが，キー制約は「ある属性が主キーと設定されたら，それがきちんと主キーの役割を果たすようデータが作られなければならない」ということを意味している．主キーを定義できれば，キー制約を定義できたようなものである．

さて，関係スキーマにおいてキー制約以外の一貫性制約には注目しない場合，関

係スキーマでは主キーは以下のように下線を引いて表す．

$$\boldsymbol{R}(\underline{A_1, A_2}, ..., A_n) \tag{3.20}$$

例えば，図 3.2 の関係「連絡先」であれば，その関係スキーマは以下のように表す．

$$\text{連絡先}\ (\underline{\text{学生}\ ID}, \text{名前}, \text{学年}, \text{大学}\ email, \text{自宅住所}) \tag{3.21}$$

Note - キーは関係スキーマに対して設定される

キーは，関係データベース内にあるタプル（行）だけを見て決めてはいけない．例えば表 3.3 に示した関係「連絡先」の場合，表の見えているところだけに注目すると，属性「名前」もキーに見える．しかし，現実的には同性同名の学生が存在しうる．そのため，「名前」をキーにすると，連絡先のタプルを一意に特定できない可能性が生じる（キーの性質を失う）．
そもそも関係データモデルにおいては，どのようなデータをデータベースに格納する可能性があるかを事前に想定して関係スキーマを設定する（その中にキーの設定も含まれる）．その上で，設定した関係スキーマに従ってデータを生成し格納する．決してインスタンスを見てキーを決めるわけではない．
キーは「**関係スキーマに対して設定される**」ことを意識しよう．

3.3.3　参照整合性制約

（関係データモデルにもとづく）関係データベースでは，対象となる事象のデータを正しく管理するために，**データを複数の関係（表）で管理する**ことがほとんどである．複合主キーの説明の例で使った関係「履修」（表 3.4 参照）には，その値だけ見ても具体的に何を意味しているのかが分からない属性「科目 ID」「学生 ID」がある．これらの意味を読み解くためには

- 「科目 ID」がどのような科目のことを指しているのか
- 「学生 ID」がどの学生のことを指しているのか

といった情報を管理する他の関係（表）が必要となる．図 3.3 は，学生の成績を管理するための関係データベースの例である．この中には関係「履修」も含まれている．図中の 3 つの関係（表）があれば，

学生

学生ID	姓	名	所属
S1	川澄	桜	A学部
S2	山畑	滝子	B学部
S3	田辺	通	C学部
	⋮		

科目

科目ID	科目名	開講年度
C1	線形代数	2023
C2	線形代数	2024
C3	統計学入門	2024
	⋮	

履修

科目ID	学生ID	成績
C1	S1	不可
C2	S1	良
C2	S2	優
C3	S3	可
	⋮	

図 3.3: 成績管理データベース

- 関係「学生」から「学生 ID」が `S1` のものを探し，
- 関係「科目」から「科目 ID」が `C1` のものを探す

ことで，関係「履修」において「学生 ID」が `S1` で「科目 ID」が `C1` の「成績」が **不可** だった件について，具体的にどんな学生が何の科目を落としてしまった（不可になってしまった）かを把握することができる．このとき，関係「履修」のタプルにある「科目 ID」の `C1` という値が，関係「科目」のインスタンスの属性「科目 ID」に含まれていなければならない（関係「科目」の科目 ID の列にその値が存在しなければならない），ということは自明であろう．関係「履修」の中にはあるが，関係「科目」には存在していなければ，データ管理として破綻していることになる．「学生 ID」の `S1` についても同様である．

上記の例では，

- 関係「履修」の属性「学生 ID」は関係「学生」の主キーである「学生 ID」と
- 関係「履修」の属性「科目 ID」は関係「科目」の主キーである「科目 ID」と

紐付いていることが分かる．言い方を変えると，関係「履修」は属性「学生 ID」および「科目 ID」の値を通して，関係「学生」および「科目」の情報を参照していることになる．

このように，関係スキーマ $\boldsymbol{R_1}(..., FK, ...)$ と $\boldsymbol{R_2}(\underline{PK}, ...)$ が与えられ，$\boldsymbol{R_1}$ におけるタプルの属性 FK の値は必ず $\boldsymbol{R_2}$ におけるいずれかのタプルの主キー PK の値と一致するように設計されているとき，$\boldsymbol{R_1}$ の属性 FK を $\boldsymbol{R_2}$ の PK に対する**外部キー（foreign key）** と呼ぶ．関係スキーマ上では，関係 $\boldsymbol{R_1}$ の属性 FK が関係 $\boldsymbol{R_2}$ の主キー PK に対する外部キーであることを以下のように記す．

$$\boldsymbol{R_1}.FK \subseteq \boldsymbol{R_2}.PK \tag{3.22}$$

図 3.3 の例では，関係「履修」の属性「学生 ID」および「科目 ID」が外部キーとなるので，関係スキーマとして

$$\text{履修.学生}\ ID \subseteq \text{学生.学生}\ ID \tag{3.23}$$
$$\text{履修.科目}\ ID \subseteq \text{科目.科目}\ ID \tag{3.24}$$

と記す．

前置きが長くなったが，本節のテーマである**参照整合性制約（referential constraint）** とは「関係スキーマにおいて外部キーが設定されたとき，そのいかなるインスタンスも設定された外部キーの条件を満たさなければいけない」という制約である．

3.3.4　データ従属性

一貫性制約として，ドメイン制約，キー制約，参照制約を挙げてきた．関係データモデルにはこれら以外にも，データ間に成立する制約を数学的に記述する手段がある．これを**データ従属性（data dependency）** と呼ぶ．

データ従属性としては，関数従属性，多値従属性，結合従属性など，様々なものが提案されている．**関数従属性（functional dependency）** は，データ従属性の中でも最も単純で，かつ実用的なものである．関数従属性は，「関係 $\boldsymbol{R}(..., X, ..., Y, ...)$ において，属性（あるいは属性集合）X の値が決まると属性 Y の値も一意に決ま

る」という性質である．このことを

$$X \to Y \tag{3.25}$$

と記す．

例えば，関係として

$$\text{連絡先}\ (\underline{\text{学生}\ ID}, \text{名前}, \text{学年}, \text{大学}\ email, \text{郵便番号}, \text{都道府県}, \text{自宅住所}) \tag{3.26}$$

が与えられたとしよう．一般常識から，住所が決まればそれが存在する都道府県や郵便番号もひとつに決まる．このことから，関係「連絡先」においては

$$\text{自宅住所} \to \text{郵便番号} \tag{3.27}$$
$$\text{自宅住所} \to \text{都道府県} \tag{3.28}$$

という関数従属性が定義できる．関数従属性はキー制約を一般化したものと捉えることができる．

データ従属性は，データの更新があったときにその影響が最小限になるように関係データベースを設計する上で，極めて重要である．それゆえ，特に関数従属性については別章で取り上げる．

クイズ

Q1. 直積

集合 S_{lang}，$S_{popularity}$，$S_{difficulty}$ を以下のように定義する：

$$S_{lang} = \{Python, R, C^{++}\} \tag{3.29}$$
$$S_{popularity} = \{\ \text{人気}, \text{不人気}\ \} \tag{3.30}$$
$$S_{difficulty} = \{\ \text{難}, \text{普通}, \text{易}\ \} \tag{3.31}$$

このとき，直積集合 $S_{lang} \times S_{popularity} \times S_{difficulty}$ の要素をすべて列挙せよ．

Q2. 関係

Q1 で定義した集合 S_{lang}，$S_{popularity}$，$S_{difficulty}$ 上の 3 項関係を適当に考えよ (※ 正解は一意に決まらないので，深く悩まずクイズに取り組むこと)．

Q3. 関係スキーマ

関係スキーマ

$$\text{学生}\ (\underline{\text{学籍番号}}, \text{氏名}, \text{学部}, \text{年齢}, \text{出身都道府県}) \tag{3.32}$$

に従う表データの例を作成せよ．なお，表の行数は見出し行を含めて 5-6 行程度でよい．

Q4. ドメイン

Q3 で定義した関係スキーマ「学生」の各属性について，そのドメインを定義せよ．

$$Dom(\text{学籍番号}) = \{...\} \tag{3.33}$$

の形式で頑張って書いてみること．

Q5. ドメイン制約

Q4 で定義した関係スキーマ（ドメイン定義を含む）に対して，ドメイン制約に違反しているタプルの例を 2，3 個列挙せよ．

Q6. 候補キー（令和 4 年度 IT パスポート試験 問 65 改題）

関係スキーマ

$$\text{従業員}\ (\text{従業員番号}, \text{従業員名}, \text{部門コード}, \text{生年月日}, \text{住所}) \tag{3.34}$$

において，候補キーは何か．なお，関係「従業員」は以下のような制約条件をもつ：

- 各従業員は重複のない従業員番号を 1 つだけもつ
- 同姓同名の従業員がいてもよい
- 各部門は重複のない部門コードを 1 つだけもつ
- 1 つの部門には複数名の従業員が所属する
- 1 人の従業員が所属する部門は 1 つだけである

Q7. 主キー

表 3.5 の「会員管理」において，想定される主キーは何か．

表 3.5: 関係「会員管理」

会員管理

店舗コード	店舗名	会員番号	会員名	会員種別
S01	星ヶ丘	1	山畑 滝子	ゴールド
S01	星ヶ丘	2	田辺 通	ゴールド
S02	八事	1	北 千種	学生
S02	八事	2	川澄 桜	プラチナ
S02	御器所	1	川澄 桜	一般

Q8. **参照整合性制約**

以下の関係スキーマをもつ 5 つの関係からなるデータベースにおいて，定義すべき参照整合性制約をあげよ．

$$\text{顧客}\ (\underline{\text{顧客}\ ID}, \text{氏名}, \text{性別}) \tag{3.35}$$

$$\text{店舗}\ (\underline{\text{店舗}\ ID}, \text{店舗名}, \text{住所}) \tag{3.36}$$

$$\text{商品}\ (\underline{\text{商品}\ ID}, \text{商品名}, \text{商品カテゴリ}\ ID, \text{単価}) \tag{3.37}$$

$$\text{購買}\ (\underline{\text{購買}\ ID}, \text{店舗}\ ID, \text{顧客}\ ID, \text{商品}\ ID, \text{個数}, \text{購買日}) \tag{3.38}$$

$$\text{商品カテゴリ}\ (\underline{\text{商品カテゴリ}\ ID}, \text{カテゴリ名}) \tag{3.39}$$

参考文献

[1] Stonebraker, M., Held, G., Wong, E. and Kreps, P.: The design and implementation of INGRES, *ACM Transactions on Database Systems*, Vol. 1, No. 3, pp. 189–222 (1976).

[2] Astrahan, M., Blasgen, M., Chamberlin, D., Eswaran, K., Gray, J., Griffiths, P., King, W., Lorie, R., McJones, P., Mehl, J., Putzolu, G., Traiger, I., Wade, B. and Watson, V.: System R: relational approach to database management, *ACM Transactions on Database Systems*, Vol. 1, No. 2, pp. 97–137 (1976).

[3] 北川博之：データベースシステム, オーム社, 改訂 2 版 (2020).

[4] 吉川正俊：データベースの基礎, オーム社 (2019).

[5] 増永良文：リレーショナルデータベース入門（第 3 版）, サイエンス社 (2017).

[6] Amazon Inc., : What is ecommerce? Definition, advantages and disadvantages of ecommerce websites and ecommerce business: `https://sell.amazon.co.jp/learn/what-is-ecommerce`.

[7] Amazon Inc., (2021), `https://press.aboutamazon.com/jp/news/`中小企業支援`/2021/10/amazon-`中小規模の販売事業者様の販売状況を発表.

[8] Codd, E. F.: A relational model of data for large shared data banks, *Communications of the ACM*, Vol. 13, No. 6, pp. 377–387 (1970).